

Angular 21 + AWS Amplify — Developer Guide

Complete technical reference guide for building Angular 21 applications with AWS Amplify, Signals, RxJS, and modern best practices. Language/framework agnostic patterns applicable to any admin dashboard or data-driven web application.

0. Git Commit Standards (Conventional Commits)

All commits must follow **Conventional Commits** format, enforced by `commitlint`. This ensures clear commit history, automated changelog generation, and semantic versioning.

Commit Format

```
<type>(<scope>): <subject>

<body (optional)>

<footer (optional)>
```

Types

- `feat` — New feature
- `fix` — Bug fix
- `refactor` — Code refactoring without feature change
- `perf` — Performance optimization
- `test` — Add or update tests
- `docs` — Documentation changes
- `chore` — Build, dependencies, tooling (no code changes affecting user)
- `style` — Code style (formatting, semicolons, etc. — no logic changes)

Scope

Feature or module affected by the commit (lowercase, no spaces):

- `auth` — Authentication
- `products` — Products module
- `dashboard` — Dashboard
- `core` — Core services
- `common` — Shared components/utilities
- `cache` — Caching service
- Combine with hyphens: `user-profile`, `product-list`

Subject Line

- **Max 100 characters** (including scope and colon)
- **Lowercase** (except proper nouns)
- **No period (.)** at the end
- **Imperative mood**: "add" not "added" or "adds"

Examples

✓ **Good Commits:**

```
git commit -m "feat(auth): implement email password login"
git commit -m "fix(cache): prevent memory leak in subscription cleanup"
git commit -m "refactor(products): simplify product filter logic"
git commit -m "perf/dashboard): optimize chart rendering with memoization"
git commit -m "test(auth): add unit tests for login validation"
git commit -m "docs(readme): update setup instructions"
git commit -m "chore(deps): upgrade angular to 21.2.4"
```

✗ Bad Commits:

```
git commit -m "Update stuff" # No type/scope
git commit -m "feat(auth): Implement email password login." # Capital letter, period
git commit -m "feat(auth-service): implemented login" # Wrong scope, past tense
git commit -m "feat(products): Add filtering and sorting and export features" # Too long
```

Multi-Line Commit (with Body & Footer)

For complex changes, add a body and footer:

```
feat(products): add product filtering and sorting

- Implement dropdown filter by category
- Add sort button for price and date
- Persist filter state to localStorage

Fixes #123
Closes #456
```

Body Guidelines:

- Explain **why** the change was made, not what
- Wrap at 72 characters
- Separate from subject with blank line

Footer Guidelines:

- Reference issues: `Fixes #123`, `Closes #456`
- Breaking changes: `BREAKING CHANGE: description`

Committing in Practice

```
# Stage changes
git add src/app/auth/login.component.ts

# Commit with type, scope, and subject
git commit -m "feat(auth): add password visibility toggle"

# For multi-line commits with body
git commit -m "feat(products): implement product search

- Add search input with debouncing
- Filter products by name in real-time
- Show result count

Fixes #789"
```

Pre-Commit Checks

The project enforces commit validation with `commitlint`:

```
# Install pre-commit hook
npx husky install

# On commit, commitlint will validate your message automatically
# If invalid, commit will be rejected with error message
```

1. Angular Version & Configuration

Package Versions

```
{
  "name": "my-admin-app",
  "version": "1.0.0",
  "dependencies": {
    "@angular/animations": "^21.2.4",
    "@angular/cdk": "^21.2.2",
    "@angular/common": "^21.2.0",
    "@angular/compiler": "^21.2.0",
    "@angular/core": "^21.2.0",
    "@angular/forms": "^21.2.0",
    "@angular/material": "^21.2.2",
    "@angular/platform-browser": "^21.2.0",
    "@angular/router": "^21.2.0",
    "@aws-amplify/ui-angular": "^5.3.2",
    "aws-amplify": "^6.16.3",
    "rxjs": "~7.8.0",
    "typescript": "~5.9.2"
  },
  "devDependencies": {
    "@angular/build": "^21.2.2",
    "@angular/cli": "^21.2.2",
    "vitest": "^4.0.8",
    "@vitest/coverage-v8": "^4.1.1",
    "angular-eslint": "21.3.0"
  }
}
```

Application Config

Bootstrapping with providers and feature initialization:

```
// src/app/app.config.ts
import { ApplicationConfig } from "@angular/core";
import { provideRouter } from "@angular/router";
import { provideHttpClient } from "@angular/common/http";
import { provideAnimationsAsync } from "@angular/platform-browser/animations/async";
import { provideLottieOptions } from "ngx-lottie";
import { provideEchartsCore } from "ngx-echarts";
import * as echarts from "echarts/core";
import { LineChart } from "echarts/charts";
import {
  GridComponent,
  TooltipComponent,
  LegendComponent,
} from "echarts/components";
import { CanvasRenderer } from "echarts/renderers";

echarts.use([
  LineChart,
  GridComponent,
  TooltipComponent,
  LegendComponent,
  CanvasRenderer,
]);

import { routes } from "../app.routes";

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(),
    provideAnimationsAsync(),
    provideLottieOptions({ player: () => import("lottie-web") }),
    provideEchartsCore({ echarts }),
  ],
};
```

3. Folder Structure Patterns

Directory Organization

```

src/app/
├─ auth/                    # Authentication module
│  ├─ constants/
│  ├─ enums/
│  ├─ login/                # Login page component
│  ├─ models/
│  └─ services/
├─ common/                  # Shared components, utilities, constants
│  ├─ components/           # Reusable UI components
│  │  ├─ custom-button/
│  │  ├─ custom-select/
│  │  ├─ custom-input/
│  │  ├─ data-table/
│  │  ├─ header/
│  │  └─ ... (other shared components)
│  ├─ constants/           # App-wide constants
│  ├─ pipes/                # Custom Angular pipes
│  ├─ services/             # Cache, upload, storage services
│  ├─ sidebar/              # Main navigation sidebar
│  ├─ utils/                # Helper functions
│  └─ models/
├─ core/                    # Core services & guards
│  ├─ auth/
│  │  ├─ guards/            # authGuard, homeGuard, roleGuard
│  │  ├─ services/         # AuthService, CurrentUserService
│  │  └─ index.ts
│  ├─ cache-manager.service.ts
│  └─ logger.service.ts
├─ products/                # Example: Products management module
│  ├─ services/             # API services, GraphQL queries
│  ├─ models/               # TypeScript interfaces
│  ├─ components/           # Feature-specific components
│  ├─ product-list.component.ts # Main list page
│  └─ product-detail.component.ts
├─ dashboard/               # Dashboard module
├─ analytics/               # Analytics module
└─ ... (other feature modules)

```

Naming Conventions

- **Components:** `ComponentNameComponent` class → `component-name.component.ts`, `.html`, `.scss`
- **Services:** `ServiceNameService` → `service-name.service.ts`
- **Models:** `EntityModel` → `entity.model.ts`
- **Utils:** `functionName()` → `function-name.util.ts`
- **Routes:** `featureName.routes.ts`
- **Pages:** `PageNamePage` component class → `page-name.component.ts`

4. Component Examples with Naming Conventions

Example 1: Login Page (Complex Component with Signals & RxJS)

```

// src/app/auth/login/login.component.ts
import { CommonModule } from "@angular/common";
import {
  ChangeDetectionStrategy,
  Component,

```

```

    OnInit,
    inject,
    signal,
} from "@angular/core";
import { toSignal } from "@angular/core/rxjs-interop";
import { FormsModule, NgForm } from "@angular/forms";
import { Router } from "@angular/router";
import { Subject, catchError, finalize, map, of, switchMap } from "rxjs";
import { AuthService } from "../services/auth.service";

@Component({
  selector: "app-login-page",
  standalone: true,
  imports: [CommonModule, FormsModule /* shared components */],
  templateUrl: "../login.component.html",
  styleUrls: ["../login.component.scss",
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class LoginPage implements OnInit {
  readonly minPasswordLength = 8;

  // Local template state
  email = "";
  password = "";

  // Reactive state (signals)
  readonly isSubmitting = signal(false);
  readonly showPassword = signal(false);
  readonly errorMessage = signal<string | null>(null);
  readonly hasSubmitted = signal(false);

  // Command stream for form submission
  private readonly submitRequests$ = new Subject<void>();

  private readonly authService = inject(AuthService);
  private readonly router = inject(Router);

  // Observable-to-Signal conversion for side effects
  private readonly submitAction = toSignal(
    this.submitRequests$.pipe(
      switchMap(() => {
        this.errorMessage.set(null);
        this.isSubmitting.set(true);
        return this.authService
          .signInWithEmailAndPassword(this.email.trim(), this.password)
          .pipe(
            switchMap(() => this.router.navigateByUrl("/dashboard")),
            map(() => null),
            catchError((error: unknown) => {
              this.errorMessage.set(this.formatAuthError(error));
              return of(null);
            }),
            finalize(() => this.isSubmitting.set(false)),
          );
      })
    ),
    { initialValue: null },
  );

  ngOnInit(): void {
    const savedMessage = this.authService.consumeErrorMessage();

```

```

    if (savedMessage) {
        this.errorMessage.set(savedMessage);
    }
}

onSubmit(form: NgForm): void {
    if (this.isSubmitting()) return;

    this.hasSubmitted.set(true);
    if (!form.valid) {
        this.errorMessage.set("Please check your email and password.");
        return;
    }

    this.submitRequests$.next(); // Trigger the submit stream
}

private formatAuthError(error: unknown): string {
    if (error instanceof Error) return error.message;
    return "Authentication failed. Please try again.";
}
}

```

Template Usage (`login.component.html`):

```

<main class="login-page">
  <div class="login-page__container">
    <h1 class="login-page__title">Sign In</h1>

    @if (errorMessage()) {
      <div class="login-page__error">{{ errorMessage() }}</div>
    }

    <form
      class="login-page__form"
      (ngSubmit)="onSubmit(loginForm)"
      #loginForm="ngForm"
    >
      <div class="form-group">
        <label for="email">Email Address</label>
        <app-text-input
          inputId="email"
          name="email"
          type="email"
          placeholder="user@example.com"
          [required]="true"
          email
          [(ngModel)]="email"
        />
        @if ((loginForm.get('email')?.touched || hasSubmitted()) &&
          loginForm.get('email')?.errors?.['required']) {
          <p class="form-group__error">Email is required.</p>
        }
      </div>

      <div class="form-group">
        <label for="password">Password</label>
        <div class="input-with-toggle">
          <app-text-input
            inputId="password"

```

```

        name="password"
        [type]="showPassword() ? 'text' : 'password'"
        placeholder="*****"
        [required]="true"
        [(ngModel)]="password"
    />
    <button
        type="button"
        class="input-with-toggle__toggle"
        (click)="showPassword.set(!showPassword())"
        [attr.aria-label]="showPassword() ? 'Hide password' : 'Show password'"
    >
        {{ showPassword() ? 'Hide' : 'Show' }}
    </button>
</div>
</div>

<button
    type="submit"
    class="login-page__submit"
    [disabled]="isSubmitting()"
>
    {{ isSubmitting() ? 'Signing in...' : 'Sign In' }}
</button>
</form>
</div>
</main>

```

Example 2: Avatar Component (Signals with `input()` & `effect()`)


```
// src/app/common/components/avatar/avatar.component.ts

import {
  ChangeDetectionStrategy,
  Component,
  effect,
  input,
  signal,
} from "@angular/core";

@Component({
  selector: "app-avatar",
  standalone: true,
  templateUrl: "./avatar.component.html",
  styleUrls: ["./avatar.component.scss"],
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class AvatarComponent {
  // Signal-based component API (inputs)
  readonly imageUrl = input<string | null>(null);
  readonly size = input<number>(40);
  readonly fallbackInitials = input<string>("??");

  // Local state
  readonly hasLoadError = signal(false);

  constructor() {
    // Reset error state when imageUrl input changes
    effect(() => {
      this.imageUrl();
      this.hasLoadError.set(false);
    });
  }

  onImageError(): void {
    this.hasLoadError.set(true);
  }
}

```

Template:

```
<div class="avatar" [style.width.px]="size()" [style.height.px]="size()">
  @if (imageUrl() && !hasLoadError()) {
    <img
      [src]="imageUrl()"
      class="avatar__image"
      loading="lazy"
      alt=""
      aria-hidden="true"
      (error)="onImageError()"
    />
  } @else {
    <div class="avatar__fallback">{{ fallbackInitials() }}</div>
  }
</div>

```

Example 3: Data List Page (Complex Page with Computed, Table & Pagination)

```
// src/app/products/product-list/product-list.component.ts

import {

```

```

    AfterViewInit,
    ChangeDetectionStrategy,
    Component,
    OnDestroy,
    OnInit,
    computed,
    effect,
    inject,
    signal,
    viewChild,
    ElementRef,
} from "@angular/core";
import { Subject } from "rxjs";
import { ProductApiService } from "../services/product-api.service";

interface ProductRow {
    id: string;
    name: string;
    price: number;
    status: string;
    createdAt: string;
}

@Component({
    selector: "app-product-list-page",
    standalone: true,
    imports: [
        /* shared components */
    ],
    templateUrl: "../product-list.component.html",
    styleUrls: ["../product-list.component.scss"],
    changeDetection: ChangeDetectionStrategy.OnPush,
})
export class ProductListPage implements OnInit, OnDestroy, AfterViewInit {
    // API state
    readonly products = signal<ProductRow[]>([]);
    readonly isLoading = signal(false);
    readonly loadError = signal<string | null>(null);
    readonly hasMore = signal(false);
    readonly nextToken = signal<string | null>(null);

    // UI state
    readonly isSidebarCollapsed = signal(false);
    readonly selectedProductIds = signal(new Set<string>());
    readonly sortField = signal<"name" | "price" | "createdAt">("createdAt");
    readonly sortDirection = signal<"asc" | "desc">("desc");
    readonly searchQuery = signal("");

    // Computed derived state
    readonly displayRows = computed(() =>
        this.products().filter((product) =>
            product.name.toLowerCase().includes(this.searchQuery().toLowerCase()),
        ),
    );

    readonly hasSelection = computed(() => this.selectedProductIds().size > 0);
    readonly selectionCount = computed(() => this.selectedProductIds().size);

    // Stream management
    private readonly loadRequests$ = new Subject<{ append: boolean }>();
    private requestId = 0;

```

```

private readonly productService = inject(ProductApiService);
private readonly infiniteScrollAnchor = viewChild<ElementRef<HTMLElement>>(
  "infiniteScrollAnchor",
);

ngOnInit(): void {
  this.loadProducts(false);
}

ngAfterViewInit(): void {
  // Set up infinite scroll intersection observer
  const element = this.infiniteScrollAnchor()?.nativeElement;
  if (element) {
    this.setupInfiniteScroll(element);
  }
}

ngOnDestroy(): void {
  // Cleanup observers and timers
}

private loadProducts(append: boolean): void {
  this.isLoading.set(true);
  const currentId = ++this.requestId;

  this.productService
    .searchProducts$(
      { search: this.searchQuery() },
      append ? this.nextToken() : undefined,
    )
    .subscribe((result) => {
      if (currentId !== this.requestId) return; // Discard stale responses

      if (append) {
        this.products.set([...this.products(), ...result.items]);
      } else {
        this.products.set(result.items);
      }

      this.nextToken.set(result.nextToken);
      this.hasMore.set(!result.nextToken);
      this.isLoading.set(false);
    });
}

private setupInfiniteScroll(element: HTMLElement): void {
  const observer = new IntersectionObserver(
    (entries) => {
      if (entries[0]?.isIntersecting && this.hasMore() && !this.isLoading()) {
        this.loadProducts(true);
      }
    },
    { threshold: 0.1 },
  );
  observer.observe(element);
}
}

```

5. Services Structure & API/GraphQL Patterns

Example 1: Auth Service (Core Service with AWS Amplify)

```
// src/app/core/auth/services/auth.service.ts
import { Injectable, inject, signal } from "@angular/core";
import { Router } from "@angular/router";
import {
  Observable,
  catchError,
  defer,
  from,
  map,
  of,
  switchMap,
  tap,
  throwError,
} from "rxjs";
import {
  fetchAuthSession,
  fetchUserAttributes,
  getCurrentUser,
  signIn,
  signOut,
} from "aws-amplify/auth";
import { Hub } from "aws-amplify/utils";

interface AuthenticatedUser {
  userId: string;
  username: string;
  groups: string[];
}

@Injectable({ providedIn: "root" })
export class AuthService {
  readonly isSignedIn = signal(false);
  readonly currentUser = signal<AuthenticatedUser | null>(null);
  readonly errorMessage = signal<string | null>(null);

  private readonly router = inject(Router);

  private readonly unsubscribe = Hub.listen("auth", ({ payload }) => {
    if (payload.event === "signedIn") {
      this.checkUserSession$().subscribe((user) => {
        if (user) void this.router.navigateByUrl("/");
      });
    }
    if (payload.event === "signedOut") {
      this.clearAuthState();
    }
  });

  // -----
  // Public API Methods (Observables)
  // -----

  signInWithEmailAndPassword$(
    email: string,
    password: string,
  ): Observable<void> {
```

```

        this.errorMessage.set(null);
        return defer(() => from(signIn({ username: email, password }))).pipe(
            tap((result) => {
                if (result.nextStep.signInStep !== "DONE") {
                    throw new Error("Multi-step authentication not supported");
                }
            }),
            map(() => undefined),
            catchError((error: unknown) => {
                this.errorMessage.set(this.formatAuthError(error));
                return throwError(() => error);
            }),
        );
    }

    checkUserSession$(): Observable<AuthenticatedUser | null> {
        return defer(() => from(fetchAuthSession({ forceRefresh: true }))).pipe(
            switchMap((session) => {
                if (!session.tokens?.idToken) return of(null);

                const groups =
                    (session.tokens.idToken.payload?.["cognito:groups"] as
                     | string[]
                     | undefined) ?? [];

                if (groups.length === 0) {
                    this.errorMessage.set("No user role is assigned to this account.");
                    return defer(() =>
                        from(signOut({ global: true })).catch(() => undefined),
                    ).pipe(map(() => null));
                }

                return defer(() => from(getCurrentUser())).pipe(
                    switchMap((authUser) =>
                        defer(() => from(fetchUserAttributes())).pipe(
                            map((attributes) => ({
                                userId: authUser.userId,
                                username:
                                    (attributes as Record<string, string>)[ "email" ] ??
                                    authUser.username,
                                groups,
                            })),
                            tap((user) => {
                                this.currentUser.set(user);
                                this.isSignedIn.set(true);
                            }),
                        ),
                    ),
                );
            }),
            catchError(() => of(null)),
        );
    }

    signOut$(): Observable<void> {
        this.errorMessage.set(null);
        return defer(() => from(signOut({ global: true }))).pipe(
            tap(() => this.clearAuthState()),
            catchError((error) => {
                this.errorMessage.set("Sign out failed. Please try again.");
                return throwError(() => error);
            })
        );
    }

```

```

    }},
  );
}

consumeErrorMessage(): string | null {
  const message = this.errorMessage();
  this.errorMessage.set(null);
  return message;
}

private clearAuthState(): void {
  this.isSignedIn.set(false);
  this.currentUser.set(null);
}

private formatAuthError(error: unknown): string {
  if (error instanceof Error) return error.message;
  return "Authentication failed. Please try again.";
}
}

```

Example 2: Data API Service (GraphQL Query Service)

```

// src/app/products/services/product-api.service.ts
import { Injectable, inject } from "@angular/core";
import { Observable, catchError, defer } from "rxjs";
import { generateClient } from "aws-amplify/api";
import { LoggerService } from "../../../core/logger.service";

interface ProductItem {
  id: string;
  name: string;
  price: number;
  status: string;
  createdAt: string;
}

interface SearchResult<T> {
  items: T[];
  nextToken: string | null;
  total: number;
}

const searchProductsQuery = /* GraphQL */ `
query SearchProducts(
  $filter: SearchableProductFilterInput
  $nextToken: String
  $sort: [SearchableProductSortInput]
) {
  searchProducts(filter: $filter, nextToken: $nextToken, sort: $sort) {
    nextToken
    total
    items {
      id
      name
      price
      status
      createdAt
      updatedAt
    }
  }
}
`

```

```

    }
  }
};

@Injectable({ providedIn: "root" })
export class ProductApiService {
  private readonly client = generateClient();
  private readonly logger = inject(LoggerService);

  searchProducts$(
    filter: Record<string, unknown> = {},
    nextToken?: string,
  ): Observable<SearchResult<ProductItem>> {
    return defer(async () => {
      try {
        const result = await this.client.graphql({
          query: searchProductsQuery,
          variables: { filter, nextToken },
        });

        const typed = result.data as {
          searchProducts: SearchResult<ProductItem>;
        };
        return typed.searchProducts;
      } catch (error) {
        this.logger.error("searchProducts failed", { error });
        return {
          items: [],
          nextToken: null,
          total: 0,
        };
      }
    });
  }

  getProduct$(id: string): Observable<ProductItem | null> {
    return defer(async () => {
      try {
        const result = await this.client.graphql({
          query: `query GetProduct($id: ID!) { getProduct(id: $id) { id name price status createdAt } }`,
          variables: { id },
        });
        return (result.data as { getProduct: ProductItem }).getProduct;
      } catch (error) {
        this.logger.error("getProduct failed", { error, id });
        return null;
      }
    });
  }
}

```

Example 3: Cache Service (Generic In-Memory/Storage Cache)

```

// src/app/common/services/cache.service.ts
import { Injectable, inject, PLATFORM_ID } from "@angular/core";
import { isPlatformBrowser } from "@angular/common";

type StorageType = "local" | "session";

export const PRODUCT_IMAGE_CACHE_KEY = "productImageCache";

```

```

export const USER_PROFILE_CACHE_KEY = "userProfileCache";

interface CacheEntry<T> {
  value: T;
}

@Injectable({ providedIn: "root" })
export class CacheService {
  public inAppCache: Record<string, { value: unknown }> = {};
  private readonly platformId = inject(PLATFORM_ID);

  // -----
  // Browser Storage (localStorage/sessionStorage)
  // -----

  private getStorage(storageType: StorageType): Storage | null {
    if (!isPlatformBrowser(this.platformId)) return null;
    return storageType === "local" ? localStorage : sessionStorage;
  }

  get<T>(key: string, storageType: StorageType = "local"): T | null {
    const storage = this.getStorage(storageType);
    if (!storage) return null;

    const cachedRaw = storage.getItem(key);
    if (cachedRaw) {
      try {
        const cached: CacheEntry<T> = JSON.parse(cachedRaw);
        return cached.value;
      } catch {
        this.remove(key, storageType);
      }
    }
    return null;
  }

  set<T>(key: string, value: T, storageType: StorageType = "local"): void {
    const storage = this.getStorage(storageType);
    if (!storage) return;

    const entry: CacheEntry<T> = { value };
    try {
      storage.setItem(key, JSON.stringify(entry));
    } catch {
      // Ignore storage write failures (quota exceeded, private mode, etc.)
    }
  }

  remove(key: string, storageType: StorageType = "local"): void {
    const storage = this.getStorage(storageType);
    if (!storage) return;
    storage.removeItem(key);
  }

  // -----
  // In-App Memory Cache (survives navigation, lost on refresh)
  // -----

  getInApp<T>(key: string): T | null {
    const entry = this.inAppCache[key];
    if (!entry) return null;
  }

```



```
        return entry.value as T;
    }

    setInApp<T>(key: string, value: T): void {
        this.inAppCache[key] = { value };
    }

    removeInApp(key: string): void {
        delete this.inAppCache[key];
    }
}
```

6. Constants Organization

Global Constants

```
// src/app/common/constants.ts

// Cache configuration
export const IMAGE_CACHE_DURATION_MS = 10 * 60 * 1000; // 10 minutes
export const IMAGE_CACHE_MAX_RETRIES = 3;
export const IMAGE_CACHE_RETRY_DELAY_MS = 1000;

// API configuration
export const API_REQUEST_TIMEOUT_MS = 30 * 1000; // 30 seconds
export const API_MAX_PAGE_SIZE = 100;

// Pagination
export const DEFAULT_PAGE_SIZE = 20;
```

Feature-Specific Constants

```
// src/app/auth/constants/auth.constants.ts
export const MIN_PASSWORD_LENGTH = 8;
export const MAX_EMAIL_LENGTH = 255;

// src/app/products/constants/product.constants.ts
export const PRODUCT_STATUS = {
    ACTIVE: "active",
    ARCHIVED: "archived",
    DRAFT: "draft",
} as const;
```

Design Tokens (CSS Custom Properties)

```
// src/styles.scss
:root {
  --color-primary: #007acc;
  --color-secondary: #6c757d;
  --color-accent: #ffb414;
  --color-success: #28a745;
  --color-danger: #dc3545;
  --color-warning: #ffc107;
  --color-surface: #131515;
  --color-surface-2: #1e2121;
  --color-surface-overlay: rgba(19, 21, 21, 0.75);
  --panel-width: 393px;
  --backdrop-blur: blur(16px);
}

html {
  background-color: #111212;
}

body {
  color-scheme: dark;
  background-color: var(--color-surface, #131515);
  color: #ffffff;
  margin: 0;
  height: 100%;
}
```

7. Utils Organization

Type Validation Utilities

```
// src/app/common/utils/datetime.util.ts
export function parseISODateTime(
  value: string | null | undefined,
): Date | null {
  if (!value) return null;

  const date = new Date(value);
  if (Number.isNaN(date.getTime())) {
    return null;
  }

  return date;
}

export function formatDate(date: Date, locale = "en-US"): string {
  return new Intl.DateTimeFormat(locale, {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
  }).format(date);
}

export function getEpochMs(value: string | null | undefined): number | null {
  const date = parseISODateTime(value);
  return date ? date.getTime() : null;
}
```

Directory Structure

```
src/app/common/utils/
├─ datetime.util.ts      # Date/time parsing & formatting
├─ array.util.ts         # Array helper functions
├─ string.util.ts        # String utilities
└─ type-guards.util.ts   # TypeScript type guards
```

8. Auth Implementation

Auth Routes

```
// src/app/auth/auth.routes.ts
import { Routes } from "@angular/router";

export const authRoutes: Routes = [
  {
    path: "login",
    loadComponent: () =>
      import("./login/login.component").then((m) => m.LoginPage),
  },
  {
    path: "signup",
    loadComponent: () =>
      import("./signup/signup.component").then((m) => m.SignupPage),
  },
  {
    path: "",
    redirectTo: "login",
    pathMatch: "full",
  },
];
```

Auth Guards

```
// src/app/core/auth/guards/auth.guard.ts
// Checks if user is authenticated
export const authGuard = (...) => { /* ... */ };

// src/app/core/auth/guards/home.guard.ts
// Redirects to home if already authenticated
export const homeGuard = (...) => { /* ... */ };

// src/app/core/auth/guards/role.guard.ts
// Checks if user has required role/feature access
export const roleGuard = (...) => { /* ... */ };
```

Auth Models

```
// src/app/auth/models/auth.model.ts

export interface AuthenticatedUser {
  userId: string;
  username: string; // email
  groups: string[]; // user roles/groups
}

export interface UserProfile {
  id: string;
  email: string;
  firstName?: string | null;
  lastName?: string | null;
  avatar?: string | null;
  role: "admin" | "user" | "viewer";
  createdAt: string; // ISO 8601 timestamp
  lastLogin?: string | null;
}

export type AuthErrorType =
  | "InvalidCredentials"
  | "NetworkError"
  | "UnknownError";

export interface AuthError {
  type: AuthErrorType;
  message: string;
}
```

9. SCSS Organization

Design Token Variables

```
// src/styles.scss - Global design tokens
:root {
  --color-primary: #007acc;
  --color-secondary: #6c757d;
  --color-accent: #ffb414;
  --color-success: #28a745;
  --color-danger: #dc3545;
  --color-surface: #131515;
  --color-surface-2: #1e2121;
  --panel-width: 393px;
  --backdrop-blur: blur(16px);
}
```

Component SCSS Pattern (BEM Naming)

```
// src/app/common/components/avatar/avatar.component.scss
```

```
.avatar {  
  position: relative;  
  display: flex;  
  cursor: pointer;  
  
  &__image {  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    overflow: hidden;  
    border-radius: 100px;  
    aspect-ratio: 1 / 1;  
    width: 100%;  
    height: 100%;  
    border: 1px solid rgba(255, 255, 255, 0.1);  
    box-sizing: border-box;  
    position: relative;  
  
    img {  
      width: 100%;  
      height: 100%;  
      object-fit: cover;  
      display: block;  
      background: transparent;  
    }  
  }  
  
  &__fallback {  
    width: 100%;  
    height: 100%;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    background: linear-gradient(  
      135deg,  
      rgba(0, 122, 204, 0.35) 0%,  
      rgba(255, 255, 255, 0.08) 100%  
    );  
    font-family: "Inter", sans-serif;  
    font-weight: 600;  
    color: #fff;  
  }  
}
```

Shared SCSS Mixins & Utilities

```
// src/app/common/styles/_shared.scss

// Skeleton shimmer animation
%sk-shimmer {
  animation: sk-shimmer 1.6s infinite;
  background: linear-gradient(
    90deg,
    rgba(255, 255, 255, 0.05),
    rgba(255, 255, 255, 0.1),
    rgba(255, 255, 255, 0.05)
  );
  background-size: 200% 100%;
}

@keyframes sk-shimmer {
  0% {
    background-position: -200% 0;
  }
  100% {
    background-position: 200% 0;
  }
}

// Scrollbar styling
@mixin custom-scrollbar($color: rgba(255, 255, 255, 0.2)) {
  scrollbar-width: thin;
  scrollbar-color: $color transparent;

  &::-webkit-scrollbar {
    width: 8px;
    height: 8px;
  }

  &::-webkit-scrollbar-track {
    background: transparent;
  }

  &::-webkit-scrollbar-thumb {
    background: $color;
    border-radius: 4px;
  }
}
```

CSS Custom Properties Usage Throughout

All component styles reference CSS variables from `:root`:

- `var(--color-primary)` — Primary color
- `var(--color-surface)` — Dark background
- `var(--panel-width)` — Fixed width for panels
- `var(--backdrop-blur)` — Blur effect for overlays

10. SVG Icon Usage Patterns

SVG Icons Location

```
src/assets/icons/  
├─ ic_add.svg  
├─ ic_arrow_left.svg  
├─ ic_bell.svg  
├─ ic_close.svg  
├─ ic_delete.svg  
├─ ic_dropdown.svg  
├─ ic_email.svg  
├─ ic_eye.svg  
├─ ic_menu.svg  
├─ ic_search.svg  
├─ ic_upload.svg  
├─ ic_check.svg  
└─ ... (40+ icons)
```

Icon Usage in Templates

Header Component with SVG Icons:


```

<!-- src/app/common/components/header/header.component.html -->
<header class="header">
  <div class="header__title-wrapper">
    @if (backUrl()) {
      <a class="header__back-btn" [href]="backUrl()" aria-label="Go back">
        
      </a>
    }
    <h1 class="header__title">{{ title() }}</h1>
  </div>

  @if (showSearch()) {
    <div class="header__search">
      <div class="header__search-content">
        
        <input class="header__search-field" type="text" />
        @if (searchQuery() && !searchDisabled()) {
          <button class="header__clear-btn" (click)="clearSearch()">
            
          </button>
        }
      </div>
    </div>
  }
</header>

```

Item Details Panel with Icons:

```

<!-- src/app/products/product-detail/product-detail.component.html -->
<div class="panel" role="dialog" aria-modal="true">
  <div class="panel__close-row">
    <button class="panel__close-btn" (click)="close()" aria-label="Back">
      
    </button>
  </div>

  <div class="panel__content">
    @if (selectedItemIds().size > 0) {
      <button class="panel__delete-btn" (click)="deleteSelected()">
        
        Delete Selected
      </button>
    }
  </div>
</div>

```

Best Practices for SVG Icons

1. **Accessibility:** Include `alt=""` and `aria-hidden="true"` for purely decorative icons
2. **Dimensions:** Always specify `width` and `height` attributes
3. **Paths:** Use consistent `assets/icons/` path for all SVG references
4. **Naming:** Follow `ic_<name>.svg` convention
5. **Styling:** Icon sizes controlled via `width` / `height` attributes or CSS

11. Signal Usage in Components

Signal Pattern 1: Local Component State

```

// Basic signals for template binding
readonly isSubmitting = signal(false);
readonly isEditing = signal(false);
readonly selectedItemId = signal<string | null>(null);
readonly errorMessage = signal<string | null>(null);
readonly hasSubmitted = signal(false);

```

In Template:

```

@if (!isEditing()) {
  <button [disabled]="isSubmitting()">Submit</button>
}

```

Signal Pattern 2: Signal Inputs & Outputs

```

// Avatar Component – Signal-based component API
export class AvatarComponent {
  // Inputs (read-only signals from parent)
  readonly imageUrl = input<string | null>(null);
  readonly size = input<number>(40);
  readonly fallbackInitials = input<string>("");

  // Local state
  readonly hasLoadError = signal(false);

  // Effect to react to input changes
  constructor() {
    effect(() => {
      this.imageUrl();
      this.hasLoadError.set(false);
    });
  }
}

// Button Component – Signal-based outputs
export class AppButtonComponent {
  readonly variant = input<"primary" | "secondary">("primary");
  readonly disabled = input(false);
  readonly isLoading = input(false);
  readonly clicked = output<void>();

  onClick(): void {
    if (this.disabled() || this.isLoading()) return;
    this.clicked.emit();
  }
}

```

Signal Pattern 3: Computed Derived State

```

// List page – computed filtered/mapped data
readonly displayItems = computed(() =>
  this.items().filter((item) =>
    item.name.toLowerCase().includes(this.searchQuery().toLowerCase()),
  ),
);

readonly itemCount = computed(() => this.displayItems().length);

readonly hasSelection = computed(() => this.selectedIds().size > 0);

// In template, automatically reruns when dependency signals change
@for (item of displayItems(); track item.id) {
  <div>{{ item.name }}</div>
}

```

Signal Pattern 4: Observable-to-Signal via `toSignal()`

```

// Component – convert Observable stream to signal
private readonly submitAction = toSignal(
  this.submitRequests$.pipe(
    switchMap(() => {
      this.errorMessage.set(null);
      this.isSubmitting.set(true);

      return this.apiService.submitData$(this.formData()).pipe(
        tap(() => this.resetForm()),
        catchError((error: unknown) => {
          this.errorMessage.set(this.formatError(error));
          return of(null);
        }),
        finalize(() => this.isSubmitting.set(false)),
      );
    }),
    { initialValue: null },
  );

// Trigger: call next() on subject
onSubmit(): void {
  this.submitRequests$.next();
}

```

Signal Pattern 5: Immutable Updates for Collections

```

// Managing item selection
readonly selectedIds = signal(new Set<string>());

toggleItemSelection(id: string): void {
  const next = new Set(this.selectedIds()); // Immutable copy
  if (next.has(id)) {
    next.delete(id);
  } else {
    next.add(id);
  }
  this.selectedIds.set(next); // Always set the entire new Set
}

// Managing list updates
readonly items = signal<Item[]>([]);

addItem(item: Item): void {
  this.items.set([...this.items(), item]);
}

removeItem(id: string): void {
  this.items.set(this.items().filter((item) => item.id !== id));
}

updateItem(id: string, updates: Partial<Item>): void {
  this.items.set(
    this.items().map((item) =>
      item.id === id ? { ...item, ...updates } : item,
    ),
  );
}

```

Signal Pattern 6: Effect Cleanup & Dependencies

```
export class DataDetailComponent {
  readonly itemId = input<string | null>(null);
  readonly descExpanded = signal(false);
  readonly descIsLong = signal(false);

  private readonly descRef =
    viewChild<ElementRef<HTMLParagraphElement>>("descEl");
  private descResizeObserver?: ResizeObserver;

  constructor() {
    // Effect 1: Reset UI when itemId changes
    effect(() => {
      this.loadItemData(this.itemId());
      this.descExpanded.set(false);
    });

    // Effect 2: Set up ResizeObserver when element is available
    effect(() => {
      const el = this.descRef()?.nativeElement;
      this.descResizeObserver?.disconnect(); // Cleanup old observer

      if (el) {
        this.descResizeObserver = new ResizeObserver(() => {
          const lineHeight = parseFloat(getComputedStyle(el).lineHeight) || 20;
          const isLong = el.scrollHeight > lineHeight * 2 + 2;
          if (isLong !== this.descIsLong()) this.descIsLong.set(isLong);
        });
        this.descResizeObserver.observe(el);
      }
    });
  }

  ngOnDestroy(): void {
    this.descResizeObserver?.disconnect(); // Final cleanup
  }
}
```

Key Takeaways

1. **Use signals for all component state** — Replace `@Input` / `@Output` with `input()` / `output()`
2. **Keep async operations in RxJS Observables** — Use `toSignal()` to convert to signals when needed
3. **Prefer `computed()` for derived state** — Automatically tracks dependencies and updates
4. **Always use `ChangeDetectionStrategy.OnPush`** — Better performance with signals
5. **Use `effect()` for side effects** — Respond to signal changes automatically
6. **Keep CSS in custom properties** — Reference design tokens from `:root`
7. **Structure services with Observable APIs** — Services return Observables, components consume via `toSignal()`
8. **Use base classes for shared logic** — Extend classes for reusable component patterns
9. **Cache data strategically** — Use `CacheService` for API responses, images, and frequently accessed data
0. **Always use standalone components** — No NgModules; compose with imports array